Laboratorio 1

Michelle Sayas y Tulio Cordero

Agosto 2026

1 ¿Cómo se implementa la recursividad en MIPS32? ¿Qué papel cumple la pila ($sp)?

En MIPS32, la recursividad no es una característica nativa de alto nivel, sino que se construye manualmente gestionando el flujo de control y la memoria. Se implementa utilizando la instrucción jal (Jump and Link), que salta a la etiqueta de la función y guarda la dirección de retorno en el registro $ra.

El papel de la pila ($sp): La pila (Stack Pointer) es fundamental e indispensable en la recursividad. Dado que solo hay un registro $ra , si una función se llama a sí misma sin guardar el $ra anterior, la dirección de retorno original se perdería, creando un bucle infinito o un error de salto al intentar volver. En el algoritmo de paridad recursivo, la pila cumple dos funciones críticas:

Preservar el contexto de retorno: Se guarda $ra para saber a dónde volver después de la llamada recursiva.

Preservar argumentos: Se guarda $a0 (el número actual n) porque cada llamada recursiva modifica este registro.

2 ¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?

Existen dos tipos principales de riesgos de desbordamiento en estos códigos:

2.1 Desbordamiento de Pila (Stack Overflow):

Riesgo: En la versión recursiva, cada número n añade un marco de pila de 8 bytes. Si el usuario ingresa un número muy grande (ej. 1,000,000), el programa intentará reservar 8 MB de pila. Dado que el tamaño de la pila es limitado en simuladores como MARS y en hardware real, el puntero $sp podría invadir el segmento de datos estáticos o el segmento de texto (código), causando un error fatal.

Mitigación: Validar la entrada del usuario para asegurar que esté dentro de un rango seguro o utilizar optimización de tail recursion (aunque en este caso específico, la lógica 1 - resultado ocurre después de la llamada, impidiendo una optimización de cola simple).

2.2 Desbordamiento Aritmético (Arithmetic Overflow):

Riesgo: Al restar números muy grandes negativos.

Mitigación: El código utiliza subu (Subtract Unsigned) para la manipulación del puntero de pila (subu $sp, $sp, 8). subu no genera excepciones por desbordamiento, lo cual es estándar para aritmética de direcciones. Si se usara sub y hubiera un desbordamiento, el programa se detendría por una excepción.

3 Diferencias entre implementación iterativa y recursiva (Memoria y Registros)

Característica	**Enfoque Iterativo** (`paridad_iterativa.`)	**Enfoque Recursivo** (`paridad_recursivo.`)
Uso de Memoria (RAM)	Constante $O(1)$. No utiliza la pila para la lógica del algoritmo. Solo usa memoria para las cadenas de texto (`.data`).	Lineal $O(n)$. El uso de memoria crece proporcionalmente al número ingresado. Por cada unidad de n, consume 8 bytes de RAM.
Uso de Registros	Utiliza registros temporales (`$t0`, `$t1`, `$t2`) que se sobrescriben en cada ciclo. No necesita preservarlos.	Requiere guardar y restaurar registros (`$a0`, `$ra`) en la RAM constantemente. Usa `$v0` para pasar valores de retorno entre niveles.
Sobrecarga (Overhead)	Mínima. Solo instrucciones de salto (`j`, `beq`) y resta.	Alta. Instrucciones extra para gestionar la pila (`sw`, `lw`, `subu`, `addu`) y saltos (`jal`, `jr`).

4 Diferencias entre ejemplos académicos y un ejercicio operativo en MIPS32

Los ejemplos de libros de texto suelen centrarse únicamente en la lógica del algoritmo (el procedimiento paridad:), asumiendo que los valores ya están en los registros correctos. Las diferencias encontradas en estos archivos operativos son:

4.1 Interacción con el Sistema (Syscalls):

Los archivos incluyen la lógica completa de entrada/salida (li $v0, 4 para imprimir, li $v0, 5 para leer enteros). Un ejemplo académico rara vez incluye el prompt o la lectura de teclado.

4.2 Segmentación de Datos:

Se define explícitamente la sección .data con cadenas ASCIIZ para hacer el programa amigable al usuario.

4.3 Gestión de Terminación:

Se incluye la llamada al sistema 10 (li $v0, 10) para terminar el programa limpiamente, evitando que la ejecución "caiga" en memoria no válida.

4.4 Manejo del "Main":

Existe una etiqueta main que actúa como orquestador, llamando a la función y luego procesando el resultado para imprimir "Par" o "Impar" basado en el retorno.

5 Tutorial de ejecución paso a paso en MARS

Para ejecutar estos códigos en el simulador MARS:

5.1 Carga:

Abrir MARS, ir a File - Open y seleccionar paridad_recursivo.asm.

5.2 Ensamblado:

Presionar F3 o el icono del destornillador/llave inglesa ("Assemble"). Esto carga el código en el segmento de texto y las variables en el segmento de datos.

5.3 Ejecución Paso a Paso:

Presione F7 (botón de "play" con un 1) para ejecutar una instrucción a la vez. Observe el registro $sp (R29): Al entrar en la función paridad, verá que su valor disminuye en 8 (hexadecimal).
Observe la Memoria: Mire la ventana "Data Segment" (seleccione "current $sp" en el desplegable). Verá cómo se apilan los valores de rayna medida quejal paridad se ejecuta repetidamente.

5.4 Entrada:

Cuando la consola de "Run I/O" pida un número, ingrese uno pequeño (ej: 3) para no hacer el seguimiento eterno.

5.5 Retorno:

Al llegar al caso base (n=0), observe cómo el programa empieza a ejecutar los lw y addu, y el registro $sp vuelve a su valor original mientras $v0 cambia entre 0 y 1.

6 Justificación de la elección del enfoque (Eficiencia y Claridad)

Elección para MIPS: Iterativo.

• Eficiencia: El enfoque iterativo es superior. Determinar la paridad es una operación trivial que, a nivel de bit, solo requiere verificar el bit menos significativo (andi $t0, $t0, 1). Incluso usando restas como en este ejercicio, el bucle while evita cientos de accesos a memoria (RAM) que realiza la versión recursiva. En MIPS, el acceso a memoria (lw/sw) es mucho más lento que las operaciones de registro.
• Claridad: Aunque la recursividad es matemáticamente elegante ($P(n) = 1 - P(n-1)$), en ensamblador añade una capa de complejidad "sucia" debido a la gestión manual de la pila. El código iterativo es más corto, más fácil de leer y menos propenso a errores humanos (bugs) relacionados con el manejo de $sp y $ra.

7 Análisis y Discusión de los Resultados:

Al ejecutar ambos programas con el número 3: 1. Recursivo:
o Apila 3, luego 2, luego 1.
o Llega a 0 (Base, retorna 0).
o Desapila 1: 1 - 0 = 1.
o Desapila 2: 1 - 1 = 0.
o Desapila 3: 1 - 0 = 1.

o Resultado final: 1 (Impar). Correcto.

2. Iterativo: o Inicializa resultado = 0.
o Ciclo n=3: resultado = 1 - 0 = 1.
o Ciclo n=2: resultado = 1 - 1 = 0.
o Ciclo n=1: resultado = 1 - 0 = 1.
o Resultado final: 1 (Impar). Correcto.

Ambos algoritmos funcionan lógicamente igual, invirtiendo el estado de paridad n veces. Sin embargo, la implementación recursiva en ensamblador demuestra el alto costo de la abstracción. Mientras que en lenguajes de alto nivel la recursividad simplifica la sintaxis, en bajo nivel (MIPS) expone la complejidad de la gestión de memoria, haciendo evidente por qué los bucles son preferibles para tareas lineales simples.